Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Kısa bir dönem sonu oturumu: kod kusuru analizi sorusunun nasıl çözüleceği
  - Dersin konuları (kod kusurları, refactoring eylemleri ve Fowler'ın video kiralama örneği) önceki derste bitmişti. Bu oturumda yeni konu işlenmedi; hoca kod kusuru sorusunun çözüm yöntemini Fowler'ın ilk sürümü (`fowler_00`) üzerinde gösterdi.
  - Soru türü: uzunca bir Java kodu verilir. Kodda hangi kod kusurlarının (code smell) nerede olduğu işaretlenir, her kusur için uygulanacak refactoring eylemi yazılır ve bütün eylemlerden sonra varılacak en doğru tasarım bir UML sınıf şemasıyla gösterilir.
- Örnek kod: `fowler_00.Customer` (ekrandan, 00:10:55)
  - `Customer` sınıfında `_name` ve `Vector<Rental> _rentals` alanları, yapıcı, `addRental`, `getName` ve uzun `statement()` metodu vardır.
  - `statement()` içinde `totalAmount`, `frequentRenterPoints`, `result` yerel değişkenleri; her kiralama için `thisAmount` geçici değişkeni; film türüne (`Movie.REGULAR`, `Movie.NEW_RELEASE`, `Movie.CHILDRENS`) göre ücret hesaplayan bir `switch`; bonus puan hesabı ve makbuz satırlarının üretimi bulunur.
- Hocanın gösterdiği iki örnek işaretleme (ekrandan, 00:14:40)
  - #1: `statement()` metodu → Long Method. İşlem: Extract Method.
  - #2: döngü içindeki geçici değişken → Long Method. İşlem: Replace Temp with Query.
- Son tasarım: 9. müdahale sonundaki sınıf şeması (ekrandan, 00:17:10, slayt 323)
  - `Movie`: `- _priceCode : int; + getCharge(daysRented : int) : double; + getBonus(daysRented : int) : int`.
  - `Rental`: `- _daysRented : int; + getCharge() : double; + getBonus() : int; + getMovieTitle() : String`.
  - `Customer`: `+ statement(); + getTotalCharge() : double; + getTotalBonus() : int`.
  - İlişkiler: `Customer` 1 → * `Rental`; `Rental` → 1 `Movie`.

Hocanın Özellikle Vurguladığı Kısımlar

- Her kod kusuruna karşılık bir refactoring eylemi
  - Cevap "şuradaki şu kusur için şu eylem" biçiminde kurulmalıdır. Kusurun adı ve eylemin adı yetmez; bir iki cümlelik gerekçe de beklenir.
- Gerekçe örnekleri
  - Long Method için: metot diğerlerinden çok uzundur ve içinde birçok alt işi barındırır, parçalanması gerekir. Ayrıca çıkarılan metodun hangi sınıfa taşınmasının ya da aynı sınıfta kalmasının daha yararlı olacağı, yani niyet de yazılmalıdır.
  - Geçici değişken için: bu değişken kodun uzamasını teşvik eder; metot zaten kısaltılmaya muhtaçtır; değişkeni Replace Temp with Query ile kaldırmak metodun kısaltılmasını kolaylaştırır.
- İşaretleme tekniği
  - Kod üzerinde yalnız numaralı işaretleme yapmak, açıklamaları ayrı bir yerde "numara, kusur, eylem, gerekçe" sırasıyla yazmak okunabilirliği korur. Her şeyi kodun üstüne yazmak hem okumayı hem yazmayı zorlaştırır.
- Son hal kodla değil sınıf şemasıyla gösterilir
  - Son tasarım için kod yazılmaz; sıralama şeması (sequence diagram) da gerekmez. Sınıf şeması, metotların hangi sınıfa gittiğini imzalarıyla gösterecek ayrıntıda ve UML

gösterim kurallarına uygun çizilmelidir.

Kısa Tekrar Notları

- Kod kusuru analizi üç adımdır: kusuru yerinde işaretle, karşılık gelen refactoring eylemini gerekçesiyle yaz, son tasarımı UML sınıf şemasıyla çiz.
- `statement()` → Long Method → Extract Method.
- Geçici değişkenin teşvik ettiği uzun metot → Replace Temp with Query.
- Son tasarımda ücret ve bonus hesabı `Movie`'de (`getCharge(int)`, `getBonus(int)`), `Rental` bunları kendi `_daysRented` değeriyle çağırır, `Customer` toplamları `getTotalCharge()` ve `getTotalBonus()` ile hesaplar.
- `Rental.getMovieTitle()` sayesinde `Customer` `Movie`'ye doğrudan erişmez (katmanlı iletişim).

Detaylı Açıklamalar

- Kusur–eylem eşlemesi neden önemli
  - Kod kusuru tek başına bir teşhistir; değer, onu gideren doğru refactoring eylemini ve bu eylemin tasarımı nasıl iyileştireceğini söyleyebilmektedir. Hoca bu yüzden her işaretlemenin yanında kısa bir gerekçe ve niyet (örneğin çıkarılan metodun nereye taşınacağı) istedi.
  - `statement()` metodu hem ücret hesabını (`switch`), hem bonus puanını, hem makbuz metnini üretir. Bu yüzden ilk teşhis Long Method, ilk çözüm Extract Method'dur. Çıkarılan hesap metotlarının verisi `Rental` ve `Movie`'de olduğundan sonraki adımlarda bunlar Move Method ile o sınıflara taşınır.
  - Döngüdeki geçici değişken, değeri bir metot çağrısıyla elde edilebildiği halde ayrı bir yerel değişkende tutulduğu için metodu uzatır ve parçalanmasını zorlaştırır. Replace Temp with Query ile değişken yerine sorgu metodu çağrılır.
- Video kiralama örneğinin varılan son hali (ekrandan, 00:17:10)
  - 9. müdahale sonunda sorumluluklar veriye göre dağıtılmıştır: fiyat kodu `Movie`'de olduğundan ücret ve bonus hesabı `Movie.getCharge(daysRented)` ve `Movie.getBonus(daysRented)` metotlarındadır. `Rental`, `_daysRented` değerini bilen sınıf olarak `getCharge()` ve `getBonus()` çağrılarını `Movie`'ye iletir ve film adını `getMovieTitle()` ile sunar. `Customer` ise yalnız makbuzu (`statement()`) ve toplamları (`getTotalCharge()`, `getTotalBonus()`) üretir.
  - Aynı slayttaki sıralama şeması akışı şöyle gösterir:

```
Kullanıcı -> aCustomer : statement()
  [her kiralama için]
    aCustomer -> aRental : getMovieTitle()   ; aRental -> aMovie : getTitle()
    aCustomer -> aRental : getCharge()       ; aRental -> aMovie : getCharge(_daysRented)
    (tek kira için ücret hesabı ve makbuz satırı basımı tamamlanır)
  aCustomer -> aCustomer : getTotalCharge()
    [her kiralama için] aRental.getCharge() -> aMovie.getCharge(_daysRented)
  aCustomer -> aCustomer : getTotalBonus()
    [her kiralama için] aRental.getBonus()  -> aMovie.getBonus(_daysRented)
  (makbuz basımı tamamlanır)
```

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.